

zebra_nav

A POMDP navigation environment with a bridge / mine / detour strategy choice, built as a substrate for mechanistic interpretability

Abstract

ZEBRA_NAV is a small partially-observable navigation environment in which an agent crosses open procedural terrain to reach a narrow goal door. The direct route is blocked by obstacles the agent can deal with in three distinct ways: **bridge** a lake (place wood on water), **mine** a mountain (turn rock to grass), or **walk around** it. Impassable tree forests line the top and bottom edges so the agent is funnelled through the obstacle-filled middle, where it must repeatedly choose among the three strategies. The point of the design is *not* the navigation itself but the **internal computation it induces**: under partial observation the agent must infer the shape and type of the obstacle ahead (a *belief*) and select a crossing strategy (an *action mode*). We train two agents on the identical task — a recurrent PPO baseline and a pure-JAX DreamerV3 world model — both reaching 100% success, and use them as a testbed for probing and **steering** those belief and strategy representations in activation space. The concrete target is to make an agent that solves the task reliably but **never digs tunnels**, by suppressing the “mine” strategy through an activation-space intervention rather than an action mask.

1 Idea and motivation

We want a task that is (i) trivial to solve, so competence is not the bottleneck, yet (ii) forces a small number of *discrete, legible decisions* that depend on a *partially-observed latent* (the obstacle ahead). Such a task is an ideal substrate for mechanistic interpretability: there should be a low-dimensional *belief* subspace (is the obstacle water or rock; how wide is it) and a *strategy* subspace (bridge vs. mine vs. detour) in the agent’s recurrent state, and we can test whether those subspaces are (a) linearly decodable and (b) *causally* steerable. The headline experiment is suppression: keep the agent fully competent while removing one option (tunneling) by editing activations — the recurrent analogue of the “cheese vector” steering result on maze-solving policy networks.

2 Environment

Tiles. A 9-tile symbolic vocabulary (no decorative classes, so the decoder cannot interpolate phantom tiles):

id	tile	role	behaviour
0	GRASS	open ground	walkable (cost 1)
1	WATER	lake	cross by <i>bridging</i> (place → wood)
2	ROCK	mountain	cross by <i>mining</i> (mine → grass)
3	WOOD	placed bridge	walkable
4	TARGET	goal door	walkable; stepping on it wins
5	OOB	off-map pad	egocentric padding only
6	TREE	forest	impassable (detour only)
7	SAND	water fringe	cosmetic, walkable like grass
8	DIRT	rock fringe	cosmetic, walkable like grass

Observation (POMDP). An egocentric crop centred on the agent: $\text{minimap} \in \{0, \dots, 8\}^{V \times V}$ with $V = 21$ (off-map cells padded with OOB), plus a scalar vector $\text{scalars} \in \mathbb{R}^5 = [\text{FACING one-hot}(4), \text{step}/\text{max}]$. The agent never sees the whole map, so the size/type of an obstacle must be *inferred* as it approaches.

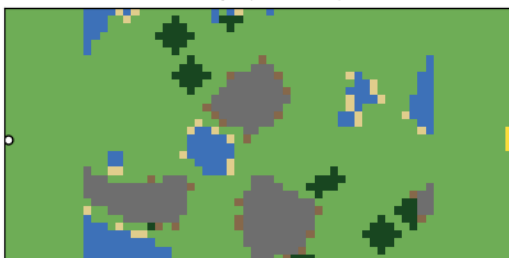
Actions. `Discrete(6)`: up / down / left / right (a move also sets `FACING`), `PLACE` (bridge the water cell in front: `WATER` $\rightarrow$ `WOOD`), and `MINE` (mine the rock cell in front: `ROCK` $\rightarrow$ `GRASS`). Trees can never be removed.

3 Task and procedural maps

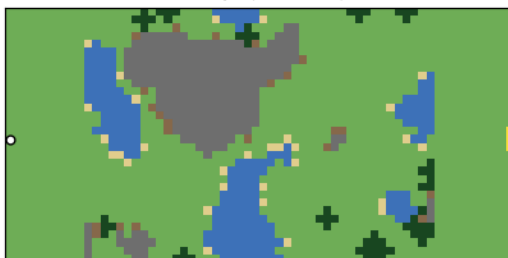
Maps are 32×64 , generated by a domain-warped fractal heightmap thresholded into lakes (`WATER`) and mountains (`ROCK`) with cosmetic `SAND`/ `DIRT` fringes; the left/right edge columns are kept obstacle-free. The **spawn** is the centre of the left edge and the **goal** is a 3-cell central door on the right wall (`goal_half = 1`). Crucially, impassable **tree forests are biased heavily toward the top and bottom rows** (about a 7:1 edge-to-middle ratio), which walls off the trivial “hug the wall to the door” route and forces traversal through the obstacle-filled centre. Figure 1 shows the curated validation set.

zebra_nav validation maps — natural 32x64

val map 0 (seed 10000)



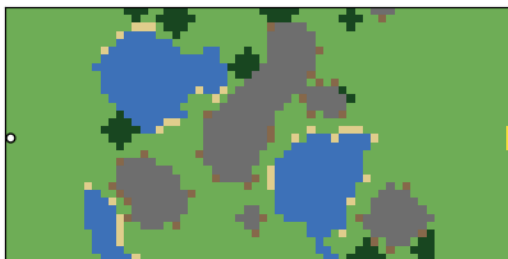
val map 1 (seed 10001)



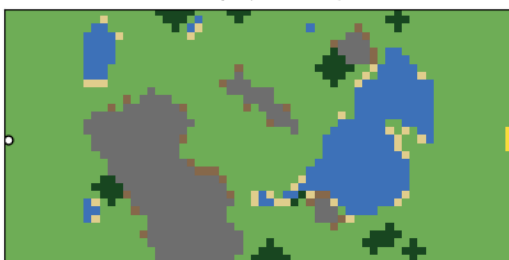
val map 2 (seed 10002)



val map 3 (seed 10003)



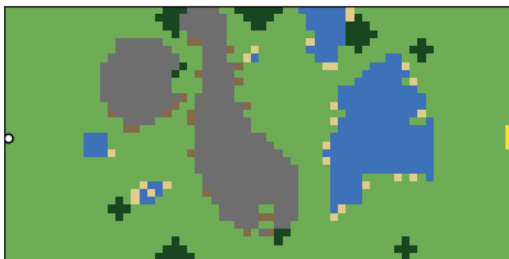
val map 4 (seed 10004)



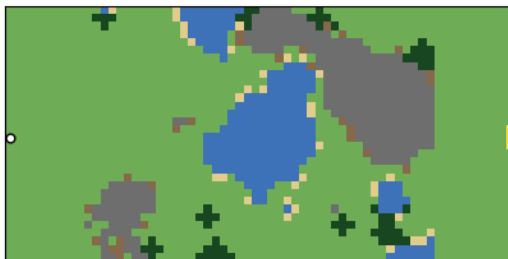
val map 5 (seed 10005)



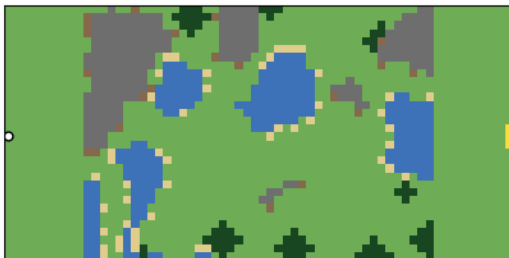
val map 6 (seed 10006)



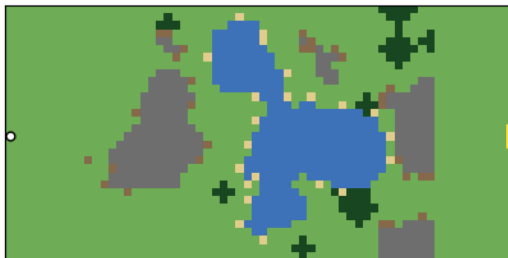
val map 7 (seed 10007)



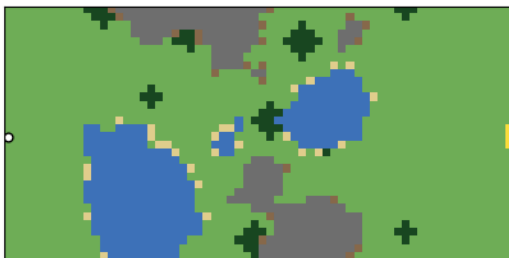
val map 8 (seed 10008)



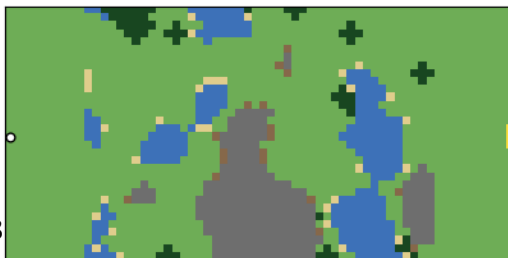
val map 9 (seed 10009)



val map 10 (seed 10010)



val map 11 (seed 10011)



val map 12 (seed 10012)



val map 13 (seed 10013)



Why this geometry. An earlier variant used the *whole* right wall as the goal; with a free top/bottom border the cheapest route is to hug an edge and sprint right, and both agents collapsed to wall-hugging with no obstacle crossing. Making the goal a narrow central door *and* foresting the edges restores the intended behaviour: the agent must go through the middle and repeatedly choose bridge / mine / detour. The strategy mix is therefore a property of the map geometry, not of the exploration schedule.

4 Reward

A move-minimising shaped reward with no per-strategy bias:

$$r_t = \underbrace{-0.01}_{\text{slack}} + \underbrace{\text{shaping}_t}_{\phi = -\text{ctg}} + \underbrace{3.0 \cdot \mathbb{I}[\text{reach door}]}_{\text{terminal}}, \quad \text{shaping}_t = 0.015 (\text{ctg}_{t-1} - \gamma \text{ctg}_t),$$

with $\gamma = 0.997$. The potential $\phi = -\text{ctg}$ uses a **min-action cost-to-go**: entering WATER/ROCK costs 2 (build + move), walkable tiles cost 1, and TREE is impassable; ctg is computed once at reset by Dijkstra seeded from the goal cell. With per-build cost 0, crossing and detouring are valued purely by total action count, so which strategy the agent picks at a given obstacle is driven by geometry (is bridging/mining shorter than going around?), exactly the decision we want to study.

5 Agents

Both agents train on the *same* task to a shared metric schema. The PyTorch PPO env (**ZebraNavEnv**) and the pure-JAX env (**ZebraNavJaxEnv**) are proven bit-for-bit equivalent (a parity test asserts identical obs / reward / termination over thousands of steps), so the DreamerV3 world model trains on exactly the task the PPO agent does.

PPO + GRU. A tile-embedding CNN with $\text{CoordConv} \rightarrow \text{MLP} \rightarrow \text{GRU}$ (128 hidden) $\rightarrow \{\text{categorical actor, value}\}$. The 128-d GRU state is the POMDP belief carrier and the primary probing target. Entropy 0.045, no annealing (keeps the stochastic policy spread out).

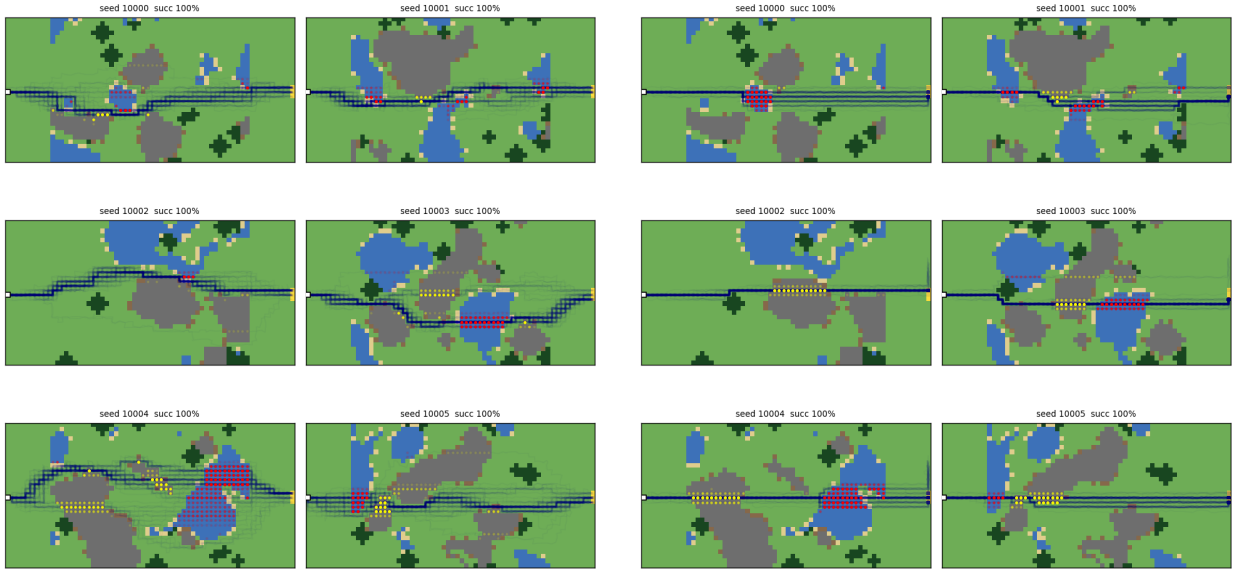
DreamerV3 (25M). The in-tree pure-JAX **purejaxwm** library: a 4-block MLP encoder/decoder on the flat $V^2 + 5$ vector obs (the minimap is symbolic, so no CNN), a block-GRU RSSM with discrete latents, TwoHot reward/value heads, RetNorm actor scaling. Paper-aligned defaults ($\gamma=0.997$, $\lambda=0.95$, $\text{lr}=4\text{e}-5$).

agent	success	episode return	episode length
PPO + GRU	100%	≈ 3.3	≈ 92
DreamerV3 (25M)	100%	≈ 3.4	≈ 95

6 Behaviour: stochastic rollouts

Figure 2 overlays 200 stochastic rollouts per map for each agent. The forests pin the paths into a central corridor; within it the agents bridge (red), mine (yellow), and detour around the larger masses. Figure 3 isolates one clean rollout of each strategy from the PPO agent — the three behaviours we want to find and manipulate internally.

atural_v2_ppo · 200 stochastic rollouts/map · success 100% · path=darkblue mine=yellow bridge=re amer_natural_v2 · 200 stochastic rollouts/map · success 100% · path=darkblue mine=yellow bridge=



(a) PPO + GRU

(b) DreamerV3

Figure 2: 200 stochastic rollouts per map. path = dark blue, bridge = red, mine = yellow. Both agents funnel through the forest-walled central corridor and cross obstacles there.

natural_centergoal3 — one stochastic rollout per strategy (blue=path, white=spawn, lime=goal, red=bridge, yellow=mine)

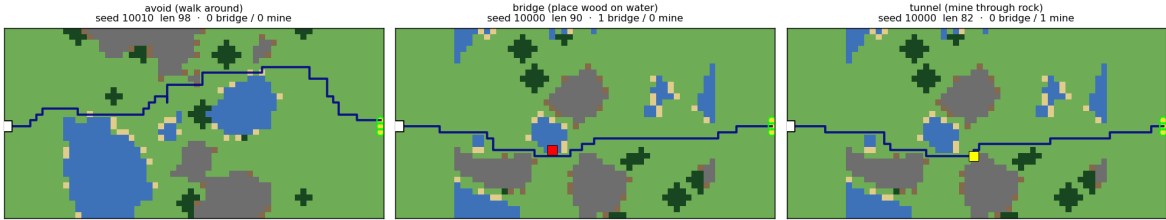


Figure 3: One stochastic rollout per strategy (PPO agent). **Avoid:** detours around a rock+lake (no build). **Bridge:** places wood to cross a lake. **Tunnel:** mines through rock. Across sampled rollouts the mix is roughly avoid:bridge:tunnel $\approx 1:1.6:0.9$, so all three modes are well represented.

7 DreamerV3 imagination

Because DreamerV3 learns a world model, we can roll it forward *in imagination* (latent rollouts under the actor, no environment) and decode each imagined latent back to an egocentric view. Figure 4 plays such a dream, decoded to Crafter sprites: warmup frames are real, then the model imagines moving and crossing obstacles, predicting coherent grass/water/rock/tree terrain.

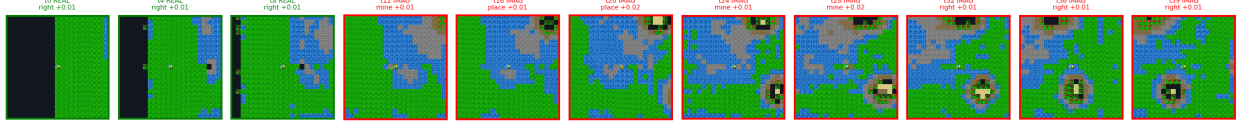


Figure 4: A DreamerV3 imagined rollout decoded to sprites (green border = real warmup, red = open-loop imagination). The world model imagines coherent terrain — including the edge tree bands — and dreams mining/placing through the middle.

An encoding caveat (relevant to probing). The Dreamer obs encodes the minimap as a single ordinal scalar per cell (`tile_id/NUM_TILES`) and the decoder *regresses* it. Reconstructions are therefore blurry and *not* a categorical tile-belief; a cleaner design (one-hot channels + per-cell softmax decoder with cross-entropy) would make tile beliefs a crisp probe target. The PPO agent already uses a proper tile *embedding*.

8 Mechanistic-interpretability plan

The agents above are the substrate. The intended programme:

1. **Activation dataset.** Fix one or two maps, sample many stochastic trajectories (logging seed + full metadata), and record the GRU state h_t (and the pre-recurrent encoder embedding) at every step, labelled with the ground-truth belief (obstacle type/size ahead) and the committed strategy (avoid / bridge / tunnel) at each obstacle.
2. **Probe.** Fit linear probes / difference-of-means directions for the belief and strategy variables; report decoding accuracy by site, and validate with unsupervised structure (PCA / clustering).
3. **Steer.** Add / clamp those directions during fresh rollouts and measure the causal behavioural change (a probe that decodes but does not steer is suspect).
4. **Suppress.** Derive the “mine” direction from the *decision* contrast (mined vs. went-around on the same rock obstacle), apply it conditionally, and verify the agent re-routes via its own bridge/detour policy rather than failing — i.e. a competent agent that **never digs tunnels**, achieved in activation space.

Reproduce. Train: `scripts/train_ppo_zebra.py --config models/zebra_nav/natural_centergoal3.yaml` and `scripts/dreamerv3_zebra_nav.py --size 25M`. Visualise: `scripts/zebra_traj_grid.py`, `scripts/zebra_s`, `scripts/viz_dreamer_zebra_imagine.py`. See `docs/zebra_nav.md` and `docs/codebase_map.md`.